MAPÚA MALAYAN DIGITAL COLLEGE
of Mapúa Malayan Colleges Laguna
In collaboration with
Arizona State University®

Security Implementation Plan

Prepared and Presented by:

Mikee Miguel
Keon Clark de Leon
Ron Carlo Dioso
Abigail Agnes

A4101 | Group 27
Bachelor of Science in Information Technology
Term 1 SY. 2026-2027

Intellectual Property Notice

This template is an exclusive property of **Mapua-Malayan Digital College** and is protected under **Republic Act No. 8293**, also known as the Intellectual Property Code of the Philippines (IP Code). It is provided solely for educational purposes within this course. Students may use this template to complete their tasks but may not **modify, distribute, sell, upload,** or **claim ownership** of the template itself. Such actions constitute copyright infringement under **Sections 172, 177, and 216** of the IP Code and may result in legal consequences. Unauthorized use beyond this course may result in legal or academic consequences.

Additionally, students must comply with the **Mapua-Malayan Digital College Student Handbook**, particularly with the following provisions:

- **Offenses Related to MMDC IT**:
    - **Section 6.2** – Unauthorized copying of files
    - **Section 6.8** – Extraction of protected, copyrighted, and/or confidential information by electronic means using MMDC IT infrastructure
- **Offenses Related to MMDC Admin, IT, and Operations**:
    - **Section 4.5** – Unauthorized collection or extraction of money, checks, or other instruments of monetary equivalent in connection with matters pertaining to MMDC

Violations of these policies may result in **disciplinary actions ranging from suspension to dismissal**, in accordance with the Student Handbook.

For permissions or inquiries, please contact MMDC-ISD at isd@mmdc.mcl.edu.ph.

TABLE OF CONTENTS

Section	
1	**Project Summary & Scope**
2	**Summary of Security Audit Findings**
3	**Finalized Controls for Implementation** Full List of Considered Controls (2–4) Selected Controls for Milestone 2 Implementation
4	**Implementation Strategy**
5	**Updated System or Workflow Diagram** Updated System Architecture Diagram Implementation Execution Workflow

1. Project Summary & Scope

Item	Details
Project Title	MotorPH Payroll Management System (Advanced OOP Version)
Brief Description	A Java enterprise application engineered to automate compensation calculations, maintain employee profiles, and generate statutory payroll summaries. The system operates on a four-tier architecture with role-based access control for HR administrators and payroll officers, enabling secure credential management, parameterized database queries, and concurrent request handling with rate limiting.
Tech Stack	Java (JDBC), MySQL, HTML/CSS/JavaScript
Deployment Environment	Local development stack with MySQL database server; production deployment to follow post-Milestone 2

2. Summary of Security Audit Findings

Category	Details
Working Controls	Basic input validation on web layer; role-based account separation (admin/user)
Broken / Missing	RISK-002 (SQL Injection): No parameterized queries; raw string concatenation in search functions. RISK-004 (Credential Exposure): Passwords stored in plaintext in database. RISK-001 (Unauthorized Access): Missing server-side session validation; parameter tampering via URL. RISK-005 (Thread Exhaustion): No request throttling; susceptible to concurrent overload attacks.
Key Risks Identified	Critical (P0): SQL Injection (RISK-002) and Plaintext Credential Storage (RISK-004). High (P1): Parameter tampering via HTTP 200 OK responses exposing payroll records (RISK-001). High (P2): Application crash under sustained concurrent load without recovery (RISK-005).

3. Finalized Controls for Implementation

3.1. Finalized Security Controls (2–4)

Control	Feasible?	Impact	Notes
Parameterized Query Execution (PreparedStatement Refactoring)	Yes	Critical	Pre-compiles SQL structures and treats user input strictly as literal parameter data, neutralizing injection attacks at the database driver boundary. OWASP ZAP verification confirms zero injection vulnerabilities detected.
Salted bcrypt Password Hashing & Externalized Configuration (.env Vault)	Yes	Critical	Isolates root database credentials from source code repositories and renders stored passwords irreversible via adaptive salted hashing. GitGuardian integration confirms zero plaintext secrets in commit history post-migration.

3.2. Implementation Tools and Methods for Finalized Controls

Control	Why Selected	Feasibility	Tools/Libraries
Parameterized Queries	Native Java JDBC support; zero external ORM dependencies; preserves existing DAO architecture; superior to Hibernate ORM which requires total refactoring.	Mode...	• java.sql.PreparedStatement
Credential Security & Config Vault	Pure Java library providing OpenBSD Blowfish salted hashing without JNI native binaries; dotenv-java parses uncommitted .env files cleanly.	Mode...	• jBCrypt 0.4 dotenv-java

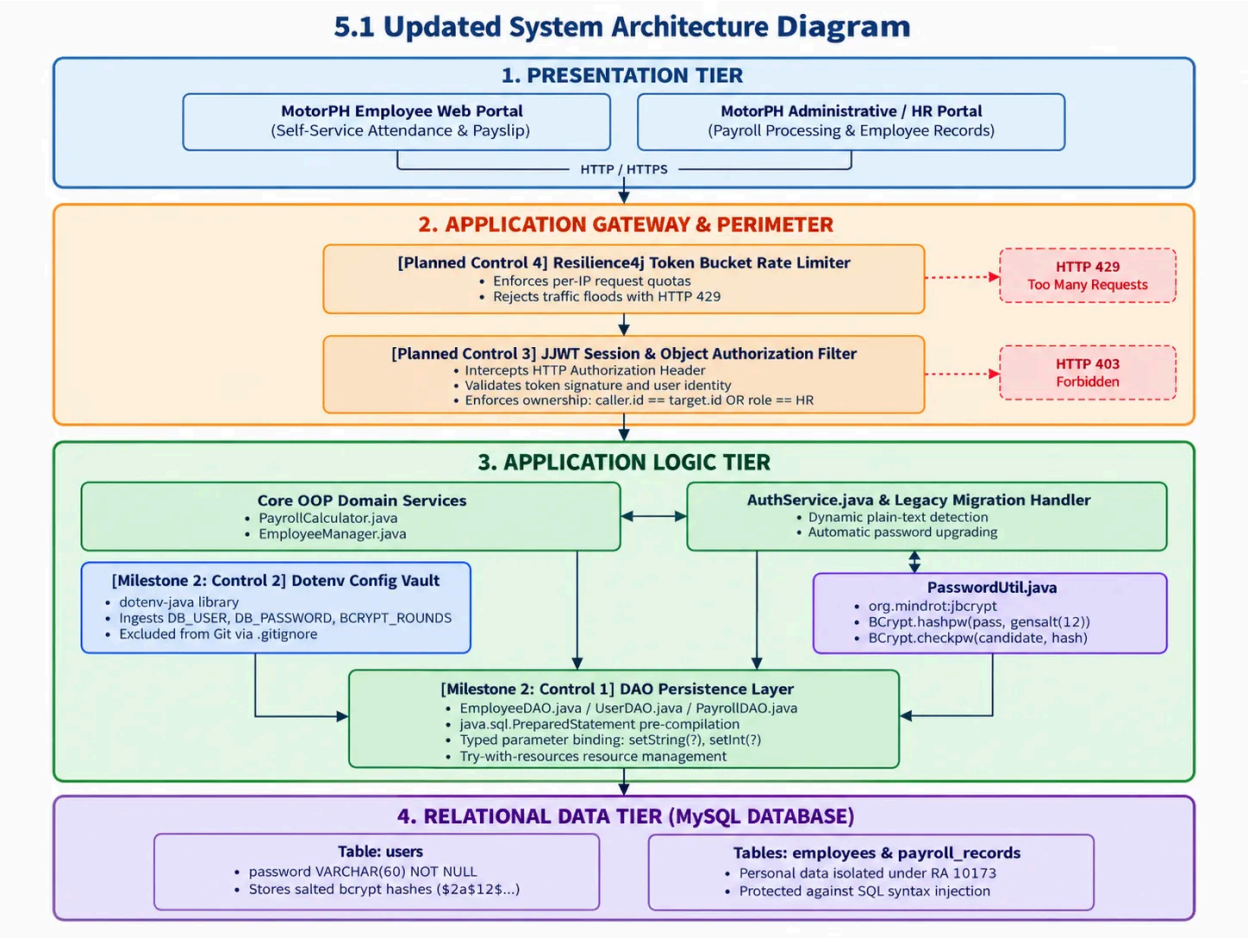
4. Implementation Strategy

Control	Integration Plan	Tool / Library	Why Chosen
Parameterized Database Query Execution	**Step 1:** Audit codebase for all java.sql.Statement instances executing raw concatenated strings. **Step 2:** Refactor methods to instantiate java.sql.PreparedStatement using SQL query templates with positional placeholders (?). **Step 3:** Bind parameters using strict type-specific setters (pstmt.setString(1, param); pstmt.setInt(2, param)). **Step 4:** Enforce try-with-resources syntax across all DAO methods to guarantee deterministic closing of database statements and result sets.]	• java.sql.PreparedStatement	• Native JDK support eliminates external dependencies • Preserves existing DAO architecture • Superior to Hibernate ORM which would require total refactoring of the data access layer.
Salted bcrypt Password Hashing & Externalized Configurati	**Step 1 Schema Preparation:** Execute DDL to modify password column capacity to VARCHAR(60) to store 60-character bcrypt	• jBCrypt 0.4 • dotenv-java	jBCrypt provides pure Java implementation of Blowfish hashing with adaptive salt generation, eliminating reliance on external cloud services (AWS

on	hashes. **Step 2: Secret Externalization**: Add dotenv-java 3.0.0 to pom.xml; create .env file in project root containing DB_URL, DB_USER, DB_PASSWORD; register .env within .gitignore [cite: 1, 2]. **Step 3: Cryptographic Utility Deployment:** Add jbcrypt 0.4 to pom.xml [cite: 1]; implement PasswordUtil.java using BCrypt.hashpw(plainText, BCrypt.gensalt(12)) and BCrypt.checkpw(candidate, hash) [cite: 1, 2]. **Step 4 Dynamic Migration Routine:** Update AuthService.java with legacy authentication handler [cite: 1, 2]. When a user logs in with a legacy plain-text password, verify the credentials immediately hash the plain text via PreparedStatement, and commit the 60-character hash to prevent user lockout [cite: 1, 2].		Secrets Manager not feasible in undergraduate lab environment); dotenv-java parses uncommitted .env files cleanly and conforms to Twelve-Factor App principles.

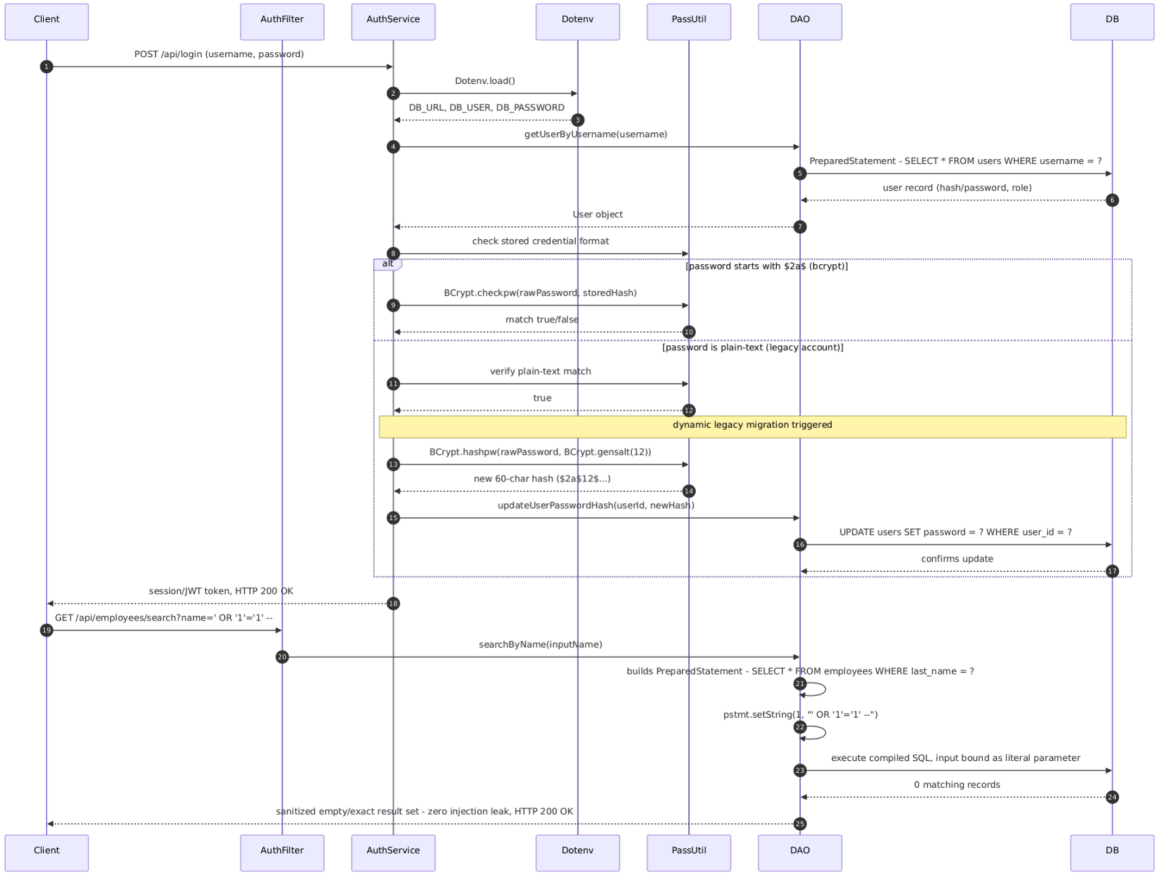
5. Updated System or Workflow Diagram

5.1 Updated System Architecture Diagram



Updated System Architecture Diagram.png

5.2 Implementation Execution Workflow



Implementation Execution Workflow.webp